Modern Operating systems

Types of operating system and justification -

Multi user OS – Uses multiple CPU's users to access system resources simultaneously. Example – UNIX, Linux.

multiprocessing OS – Uses multiple CPU's or cores to exaute process efficiently.
Linux, windows.

multitasking OS – Excutes multiple process concurrently using CPU sheduling. Ex – windows, Linux.

Timesharing OS – Allocates CPU time slice among users and processes for interactive operation.

Real-time OS – Provides responses within specified time constraints.

Suitable choice – A multi user, multitasking, multiprocessing OS with users,

system calls to create

In a UNIX / Linux system, the sequence is,

fork() – creates a child process.

exec() – loads & executes a new program

wait() – Allows the parent to wait for the child to finish.

exit() – Terminates the process.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        execl("/bin/ls", "ls", "-l", );
        exit(1);
    } else if (pid > 0) {
        wait(NULL);
    } return 0;
}
```

(2) File & Device management

Category	Purpose	Examples
file management	create, open, read, write & delete.	open(), read(), write().
Device management	Access and control I/O devices	ioctl(), read(), write().

Real-Time operating system.

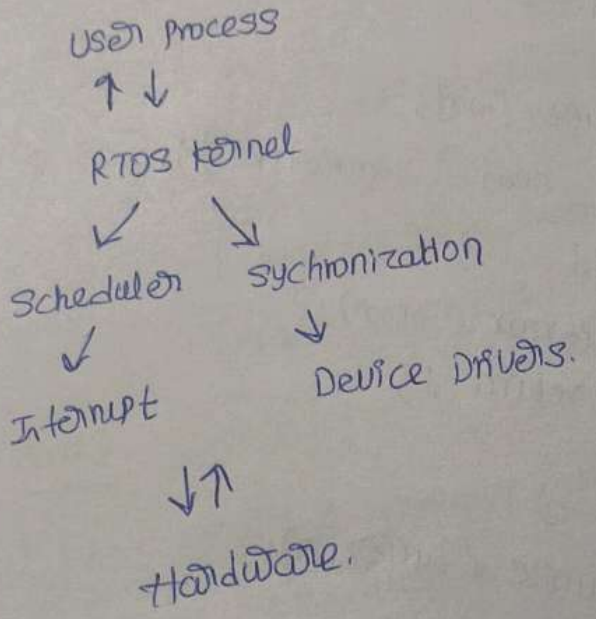
Type of OS & suitability

For embedded industrial machinery because it provides predictable scheduling and responds to events within specified deadlines.

Hard RTOS – missing a deadline is unacceptable.

Soft RTOS – occassional deadline misses.

Interaction



System calls

→ sem_wait () – Decrease a semaphore.

→ sem_post () – Increase a semaphore.

→ pthread_cond_wait() – waits for a condition.

→ pthread_mutex_unlock () – release mutex.

layered structure for file operations

User Application
↓
systemcall Interface
↓
Virtual File system
↓
File system implementation
↓
storage Device.

Program

```
#include <fcntl.h>
#include <unistd.h>
int main () {
    char buffer [100];
    int fd = open ("sample.txt");
    if (fd == -1) {
        perror ("open");
        return 1;
    }
    if (n>0)
        write (buffer, n);
        close (fd);
        return 0;
}
```

sequence :

open() → read() → write() → close().

Distributed operating systems

classification & comparision

A Distributed operating system manages multiple inter connected computers and coordinates their resources so they can operate as unified system.

Architecture of OS

[Distributed Applications / users]

↑↓

[Distributed OS services — Naming, scheduling.]

↑↓

[Communication Network NodeA, NodeB, NodeC]

System calls

→ socket() – creates a communication endpoint.

→ bind() – Associates a socket with local address.

→ connect() – Connects a client to remote server.

→ send() – Transmit & recieve the data.

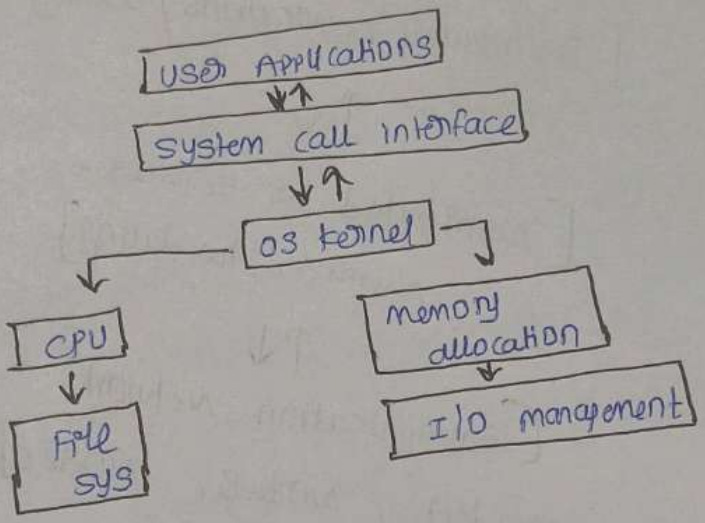
→ sendto() – Exchange datagrams using a specified address.

(5) major functions of an operating systems.

Process management
→ creates, schedules, synchronizes, and terminates the process.
→ CPU, scheduling.

Memory management
→ Allocates and dellocates memory.

Diagram :



Program :

```
#include <stdlib.h>
#include <sys/man.h>
int main() {
    void *p = malloc(1024);
    if (p == NULL) return 1;
    free(p);
    return 0;
}
```